

ANALYSIS AND DESIGN OF ALGORITHMS

Instructor: Dr. Mert Büyükdede



Master Theorem Method

The **Master Theorem** is a tool used in algorithm analysis, particularly for determining the time complexity of algorithms that follow the **divide and conquer** approach.

In divide and conquer algorithms, a problem is broken down into smaller subproblems, each of which is solved independently, and the results are combined.

To compute the time complexity of these algorithms, a **recurrence relation** is often used. The Master Theorem helps us solve these recurrence relations for certain types of problems.

The general form of the recurrence relation that the Master Theorem applies to is:

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

Master Theorem Method

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

$T(n)$ is the total time complexity of the algorithm.

a is the number of subproblems that the problem is divided into.

n/b is the size of each subproblem (this indicates how much the problem is divided).

$O(n^d)$ represents the cost of dividing and combining the results of the subproblems.



The Master Theorem classifies the solution to these recurrence relations into three cases:

Case I: The cost of solving subproblems dominates

If $a > b^d$, the time complexity is:

$$T(n) = O(n^{\log_b a})$$

In this case, the subproblems are large enough that their combined cost dominates the time complexity, and the overall complexity is determined by the number of subproblems and their size.

Case 2: The cost of solving subproblems and the cost of combining results are the same

If $a=b^d$, the time complexity is

$$T(n) = O(n^d \log n)$$

Here, the costs of solving subproblems and combining their results are similar, and the total complexity includes both factors, with a logarithmic factor arising from the number of levels of recursion.

Case 3: The cost of combining results dominates

If $a < b^d$, the time complexity is

$$T(n) = O(n^d)$$

In this case, the cost of dividing and combining the results of subproblems is larger than the cost of solving the subproblems, so the overall complexity is determined by the combining step.

The Master Theorem provides a quick way to determine the time complexity of many divide-and-conquer algorithms without needing to solve the recurrence relation step-by-step.

Master Theorem to Solve Recurrence Relations: $T(n) = a * T(n/b) + f(n)$

Master Theorem (Θ - version)

If $f(n) \in \Theta(n^d)$ where $d \geq 0$, then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Note: To satisfy the definition of a polynomial, 'd' should be a non-negative integer.

Note: To apply Master Theorem, the function **f(n)** should be a **polynomial** and **should be monotonically increasing**

If $f(n) \notin \Theta(n^d)$; but
 $f(n) \in O(n^d)$, then

$$T(n) \in \begin{cases} O(n^d) & \text{if } a < b^d, \\ O(n^d \log n) & \text{if } a = b^d, \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Master Theorem (O - version)

where $d \geq 0$ and an integer

Note: We will try to apply the Θ - version wherever possible. If the Θ - version cannot be applied, we will try to apply the **O-version**.

Master Theorem to Solve Recurrence Relations: $T(n) = a * T(n/b) + f(n)$

1) $T(n) = 4T(n/2) + n$

can be written as

$$T(n) = 4 T(n/2) + \Theta(n)$$

$$a = 4; b = 2; d = 1 \rightarrow a > b^d$$

$$T(n) = \Theta\left(n^{\log_2 4}\right) = \Theta(n^2)$$

2) $T(n) = 4T(n/2) + n^2$

Can be written as

$$T(n) = 4 T(n/2) + \Theta(n^2)$$

$$a = 4; b = 2; d = 2 \rightarrow a = b^d$$

$$T(n) = \Theta(n^2 \log n)$$

3) $T(n) = 4T(n/2) + n^3$

Can be written as

$$T(n) = 4 T(n/2) + \Theta(n^3)$$

$$a = 4; b = 2; d = 3 \rightarrow a < b^d$$

$$T(n) = \Theta(n^3)$$

4) $T(n) = 4T(n/2) + 1$

Can be written as

$$T(n) = 4 T(n/2) + \Theta(n^0)$$

$$a = 4; b = 2; d = 0 \rightarrow a > b^d$$

$$T(n) = \Theta\left(n^{\log_2 4}\right) = \Theta(n^2)$$

5) $T(n) = 4T(n/2) + (1/n)$

$$T(n) = 4T(n/2) + n^{-1}$$

$$a = 4, b = 2, d = -1 (< 0)$$

$f(n) = 1/n$ is not a polynomial.

Master Theorem cannot be applied.

Master Theorem: More Problems

$$T(n) = 3 T(n/3) + \sqrt{n}$$

We cannot write $\sqrt{n} = \Theta(n^d)$,
because $d = 1/2$ is not an integer.
Hence, we have to use the O-notation.

$\sqrt{n} = O(n)$, the smallest possible integer for which
 $\sqrt{n}$ can be written as $O(n^d)$.

$$T(n) = 3 T(n/3) + O(n)$$

$$a = 3, b = 3, d = 1$$

$$a = b^d.$$

Hence, $T(n) = O(n^d \log n) = O(n \log n)$.

Master Theorem: More Problems

$$T(n) = 4 T(n/2) + \log n$$

$\log n \notin \Theta(n^d)$, where 'd' is an integer

But, $\log n \in O(n^d)$, where $d = 1$ is the smallest possible integer for which $\log n$ can be written as $O(n^d)$

$$a = 4; b = 2; d = 1 \quad a > b^d; \text{ Hence, } T(n) = O\left(n^{\log_2 4}\right)$$
$$T(n) = O\left(n^{\log_2 4}\right) = O(n^2)$$

$$T(n) = 6 T(n/3) + n^2 \log n$$

$n^2 \log n \notin \Theta(n^d)$, where 'd' is an integer

But, $n^2 \log n \in O(n^d)$, where $d = 3$ is the smallest possible integer for which $\log n$ can be written as $O(n^d)$

$$a = 6; b = 3; d = 3 \quad a < b^d; \text{ Hence, } T(n) = O(n^3)$$

Example for Master Theorem

$$T(n) = 7T\left(\frac{n}{3}\right) + n^2$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log^2 n$$

Homework

Solve the time complexity of an algorithm with the following recurrence equation using both the Master Theorem and Substitute Method.

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

Merge Sort

Merge Sort is a sorting algorithm that uses the "Divide and Conquer" strategy. The algorithm aims to find a solution by breaking a large problem into smaller pieces. The basic steps of the Merge Sort algorithm are as follows:

Divide: It divides the input array approximately in half. This process continues until the number of elements in the array becomes 1.

Sort: Each small piece (in this case, when the number of elements in each sub-array decreases to 1) is sorted. Therefore, each sub-array becomes sorted.

Merge: These sorted small pieces are then combined into a single large sorted array. This merging process is done by comparing the elements from each sub-array from smallest to largest.



Merge sort is a widely used sorting algorithm in computer science that follows the "divide and conquer" approach. It is effective for sorting large datasets and has several real-life applications:

1.Database Management: Merge sort is used in database management systems to efficiently sort large datasets, especially when dealing with disk-based data. It is ideal for external sorting, where data does not fit entirely into memory.

2.File Systems: It is applied in file systems to sort lists of files, for example, sorting file names alphabetically or by file size.

3.Data Analysis and Reporting: When analyzing large datasets, merge sort helps sort the data according to specific criteria, improving the efficiency of data processing and reporting.

4.Internet Search Engines: Merge sort can be used to rank search results based on factors such as relevance or frequency, enabling the quick and accurate sorting of large amounts of web page data.

5.Competition and Ranking Systems: In sports events or competitions, merge sort can be used to rank participants by their scores or other metrics, ensuring an efficient and accurate ranking process.



Merge Sort

Split array $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C

- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows: – Repeat the following until no elements remain in one of the arrays:
 - compare the first elements in the remaining unprocessed portions of the arrays
 - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
- Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.



Merge Sort

ALGORITHM *Mergesort*($A[0..n-1]$)

//Sorts array $A[0..n-1]$ by recursive mergesort

//Input: An array $A[0..n-1]$ of orderable elements

//Output: Array $A[0..n-1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A)

Merge Sort

ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted

//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C

$i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow 0$

while $i < p$ **and** $j < q$ **do**

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$; $i \leftarrow i + 1$

else $A[k] \leftarrow C[j]$; $j \leftarrow j + 1$

$k \leftarrow k + 1$

if $i = p$

 copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

Incase of a tie $B[i] = C[j]$

Insert the element in the
Left sub array in A.

Explain Merge Sort Algorithm

```
MERGE_SORT(A, left, right)
    If left >= right then
        Return // Base case: If only one element remains, it's already sorted

    mid ← (left + right) / 2 // Find the middle index

    MERGE_SORT(A, left, mid) // Recursively sort the left half
    MERGE_SORT(A, mid + 1, right) // Recursively sort the right half

    MERGE(A, left, mid, right) // Merge the two sorted halves
```

Explain Merge Sort Algorithm

```
MERGE(A, left, mid, right)
    L ← A[left:mid] // Copy the left half
    R ← A[mid+1:right] // Copy the right half

    i ← 0, j ← 0, k ← left // Initialize indices

    // Compare elements from both subarrays and merge them into A
    While i < length(L) AND j < length(R) do
        If L[i] ≤ R[j] then
            A[k] ← L[i]
            i ← i + 1
        Else
            A[k] ← R[j]
            j ← j + 1
        k ← k + 1

    // Copy any remaining elements from the left subarray
    While i < length(L) do
        A[k] ← L[i]
        i ← i + 1
        k ← k + 1
```

Time Complexity of Merge Algorithm

Divide Phase: $O(\log n)$

Merge Phase: $O(n)$

Total Time Complexity: $O(n \log n)$

Merge Sort

Advantages:

- Number of comparisons performed is nearly optimal.
- For large n , the number of comparisons made by this algorithm in the average case turns out to be about $0.25n$ less and hence is also in $\Theta(n \log n)$.
- Mergesort will never degrade to $O(n^2)$
- Another advantage of mergesort over quicksort and heapsort is its stability.
- (A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input array to be sorted.)

Limitations:

The principal shortcoming of mergesort is the linear amount [$O(n)$] of extra storage the algorithm requires.

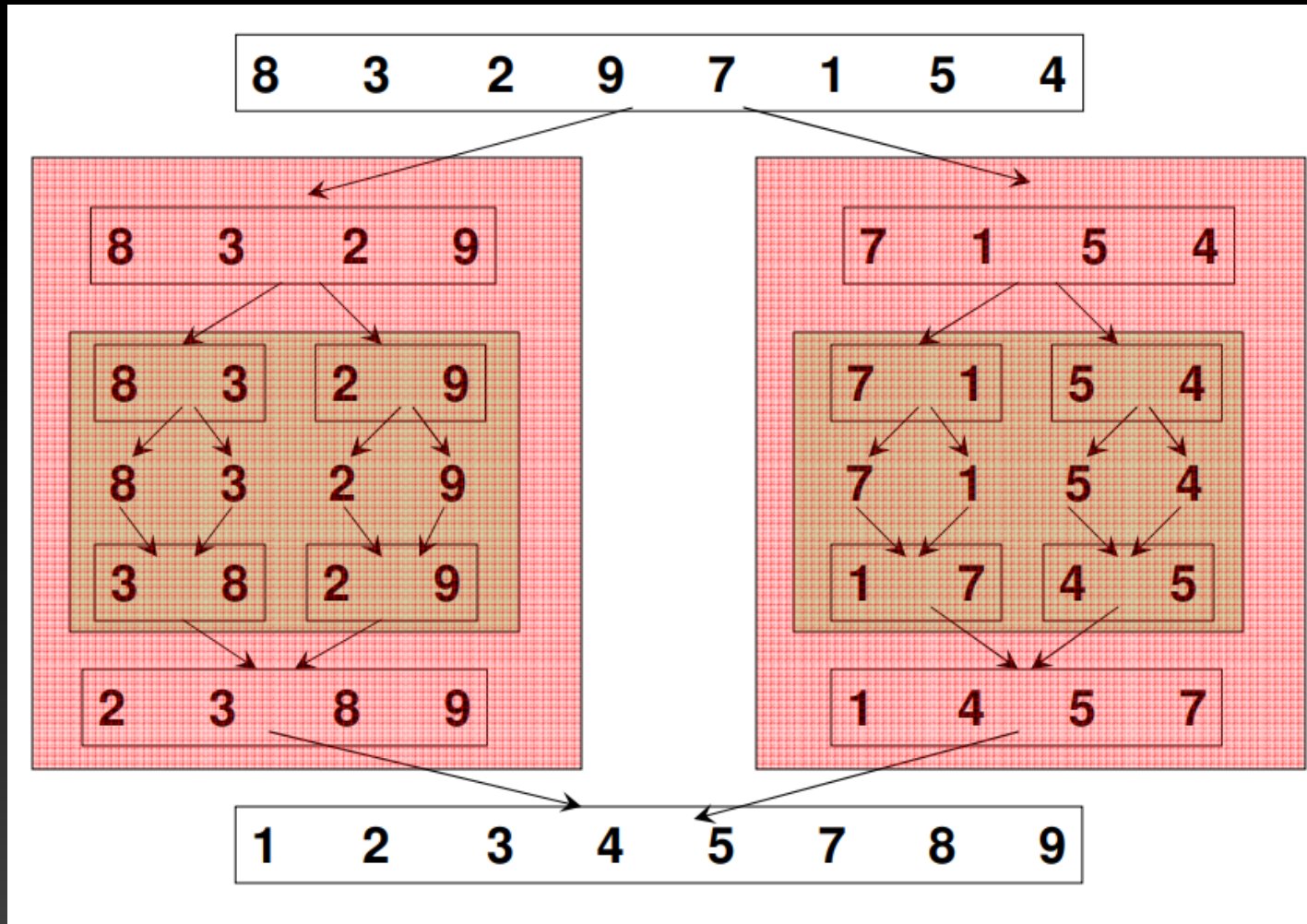
Though merging can be done in-place, the resulting algorithm is quite complicated and of theoretical interest only.

Example of Merge Sort

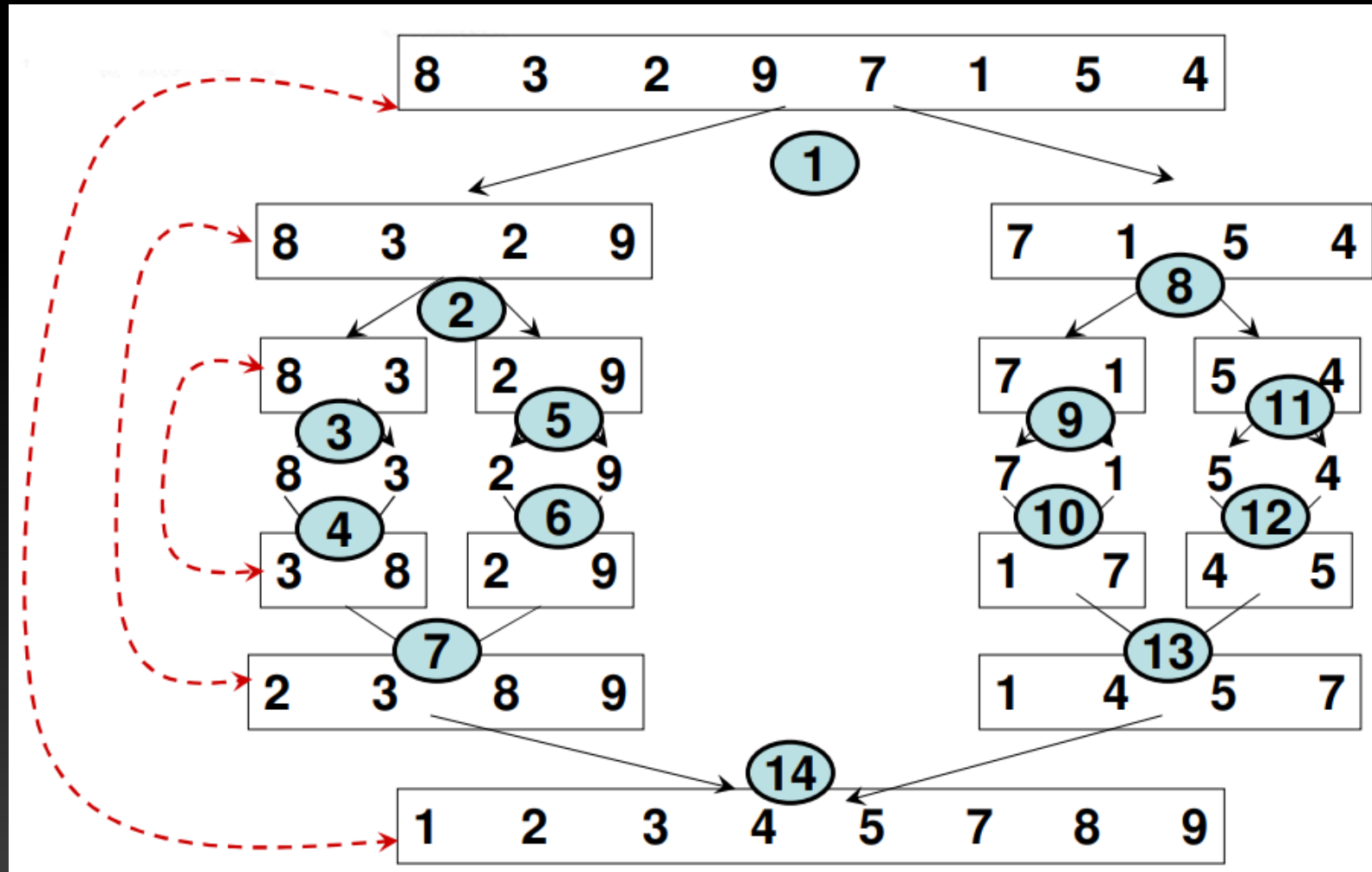
Let's sort the given array from smallest to largest.

8	3	2	9	7	1	5	4
---	---	---	---	---	---	---	---

Example of Merge Sort



The order recursion runs



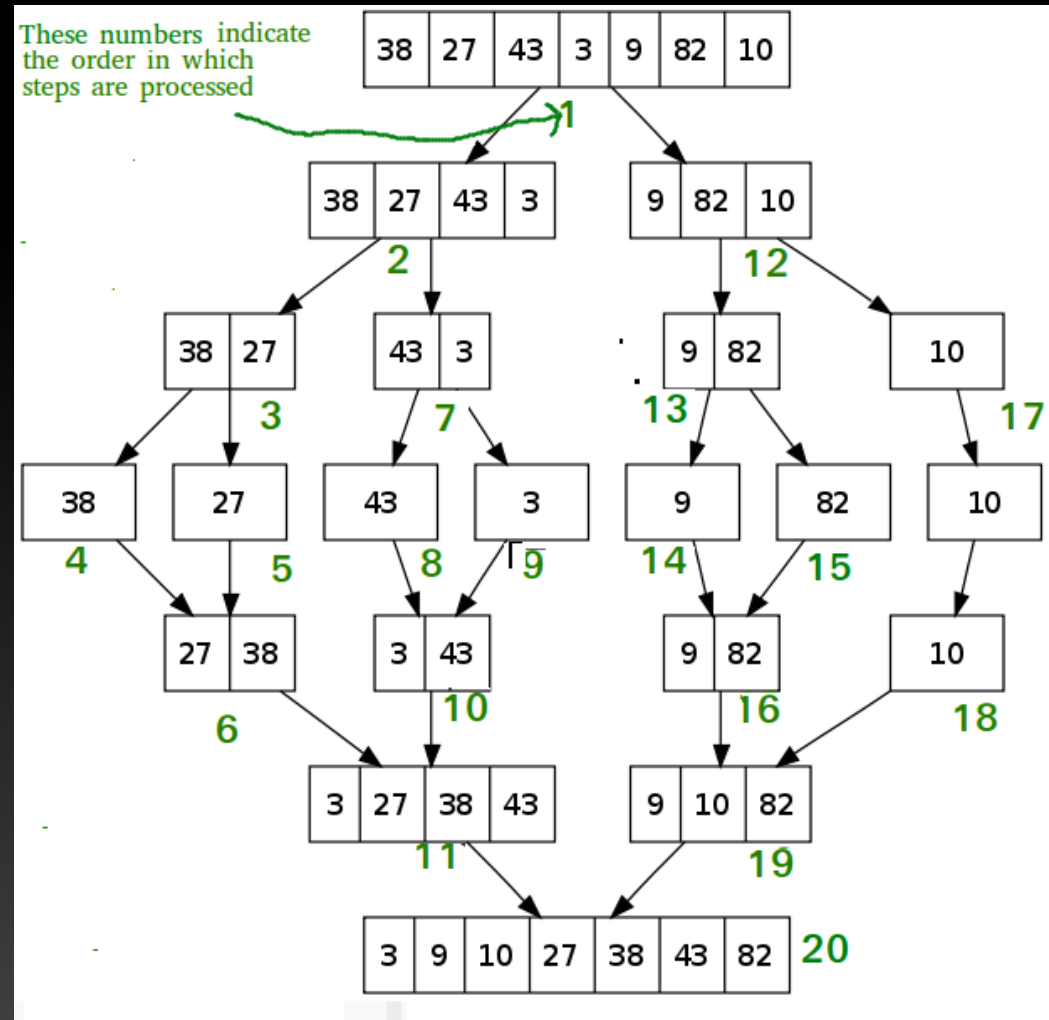
Example of Merge Sort

Sort this array given in a mixed form according to the merge sort method.

How many steps does it take to sort?

38	27	43	3	9	82	10
----	----	----	---	---	----	----

Example of Merge Sort



HOMEWORK 2

Example of Merge Sort

Sort this array given in a mixed form according to the merge sort method

How many steps does it take to sort?

Let's create java code for this sort

18	26	32	6	43	15	9	1	22	26	19	55	37	43	99	2
----	----	----	---	----	----	---	---	----	----	----	----	----	----	----	---

Analysis of Merge Sort

The recurrence relation for the number of key comparisons $C(n)$ is

$$C(n) = 2C(n/2) + C_{\text{merge}}(n) \quad \text{for } n > 1, C(1) = 0$$

At each step, exactly one comparison is made, after which the total number of elements in the two arrays still needed to be processed is reduced by one.

Best case: We will encounter $n/2$ comparisons (when every element in the left sorted sub array is less than or equal to the first element in right sorted sub array)

Worst case: We will encounter $(n-1)$ comparisons (when smaller elements come from the alternating sub arrays; neither of the two sub arrays will become empty before the other sub array contains just one element).

Analysis of Merge Sort

Though best case is different from worst case, both are $\sim n$, as n increases.
Hence, the time complexity to merge: $C_{\text{merge}}(n) = \Theta(n)$

$$C(n) = 2 * C(n/2) + \Theta(n) \text{ for } n > 1 \text{ and } C(1) = 0$$

$$a = 2; b = 2; d = 1$$

$$a = b^d$$

$$\text{Hence, } C(n) = \Theta(n \log n)$$

Merge Sort: Space-time Tradeoff

Unlike the sorting algorithms (insertion sort, bubble sort, selection sort) we saw in Module 1, Merge sort incurs a time-complexity of $\Theta(n \log n)$, whereas the other sorting algorithms we have seen incur a time complexity of $O(n^2)$ or $\Theta(n^2)$.

- The tradeoff is Merge sort requires additional space proportional to the size of the array being sorted. That is, the space-complexity of merge sort is $\Theta(n)$, whereas the other sorting algorithms we have seen incur a space-complexity of $\Theta(1)$.

Algorithms that incur a $\Theta(1)$ space complexity are said to be “in place”

HOMEWORK 3

Activity : Sorting Customer Data Using Merge Sort:A Practical Java Implementation

The dataset includes the customer's **name**, **purchase amount**, and **satisfaction score**.

Students will use the provided code to sort the customer data:

- Sort by Purchase Amount: Sort the data based on the actual spending amount.
- Sort by Satisfaction Score: Sort the data based on the customer satisfaction score.

Let's write a Java, (Python, C....) code that will sort these situations.



Number of Inversions in an Array

- Given an array A , an inversion is said to have occurred if $i < j$ and $A[i] > A[j]$.

- Example**

0	1	2	3	4	5
2	8	1	9	3	7

Inverted Pairs

(2, 1)

(8, 1)

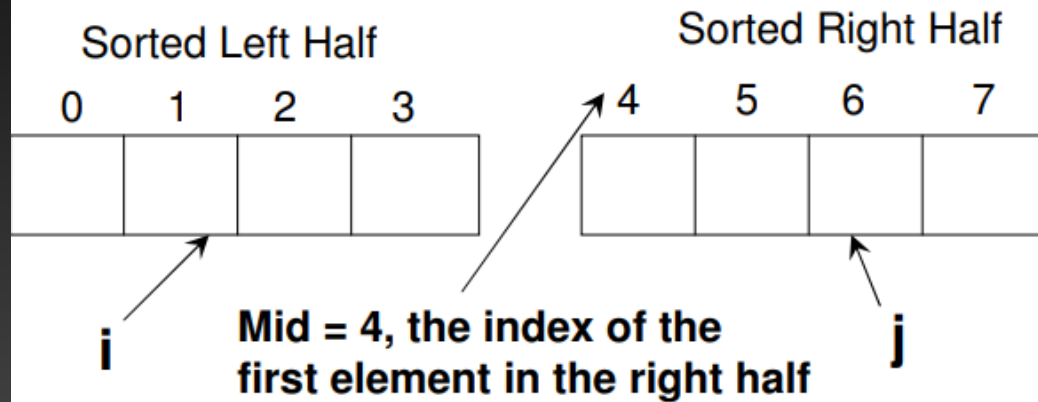
(8, 3)

(8, 7)

(9, 3)

(9, 7)

The number of inversions in an array can be computed as the Sum of the number of inversions encountered in each of the Merging steps of the Merge Sort algorithm.



If $A[i] > A[j]$, then everything to the right of Index in the sorted left half are also going to be greater than $A[j]$. Hence, the number of inversions due to $A[i] > A[j]$ is: $Mid - i$.

Inversions in the Merge Step Example 2

Mid = 5

0	1	2	3	4	5	6	7	8	9
14	17	19	22	25	13	16	18	20	27

							Sorted Array									
Index i	Index j	A[i]	A[j]	Inv Y/N	# Inv		0	1	2	3	4	5	6	7	8	9
0	5	14	13	Y	5 - 0 = 5		13									
0	6	14	16	N	-		13	14								
1	6	17	16	Y	5 - 1 = 4		13	14	16							
1	7	17	18	N	-		13	14	16	17						
2	7	19	18	Y	5 - 2 = 3		13	14	16	17	18					
2	8	19	20	N	-		13	14	16	17	18	19				
3	8	22	20	Y	5 - 3 = 2		13	14	16	17	18	19	20			
3	9	22	27	N	-		13	14	16	17	18	19	20	22		
4	9	25	27	N	-		13	14	16	17	18	19	20	22	25	
-	9	-	27	N	-		13	14	16	17	18	19	20	22	25	27

Inversions in the Merging Step

$$= 5 + 4 + 3 + 2$$
$$= 14$$